Build an Education OS architecture blueprint: map out the Student, Faculty, and Institutional Business layers into a unified super-app schema

Computer
Unified Higher-Ed CRM & SIS comparison matrix: how EAB Navigate360, Ellucian, and HEM consolidate student records, faculty workflows, and finance
Computer
Show me the complete SQL schema and API specs for all three layers
Map out the cross-layer workflows between student actions and staff tasks
Design the shared identity and RBAC authorization model for Education OS

Education OS should be designed as one unified super-app with three operational layers: **Student OS**, **Faculty/Institution OS**, and **Business OS**, all running on a shared identity, workflow, and data core. Current higher-ed platforms show a clear pattern: EAB Navigate360 concentrates CRM-style advising, coordinated care, alerts, and campaigns; Ellucian concentrates lifecycle SIS, registration, advising, and finance; HEM positions a single platform for admissions, records, academics, dashboards, and finances.[1] [2] [3]

Layered blueprint

The architecture should treat each layer as a role-specific workspace over the same institutional graph, not as separate products. Ellucian describes an end-to-end student lifecycle platform that connects student, HCM, and finance, while EAB emphasizes a shared student profile, coordinated care network, and workflow automation across campus teams.[3] [4]

Layer	Primary users	Main responsibilities
Student OS	Applicants, students, alumni	Profiles, registration, degree planning, messaging, tasks, support, portfolio
Faculty OS	Faculty, advisors, mentors, program directors	Progress reports, advising, alerts, notes, appointments, approvals
Institution OS	Admissions, registrar, student affairs, aid, operations staff	Cases, routing, workflows, records, outreach, service delivery
Business OS	Finance, provost, deans, business officers, planners	Tuition, budgeting, resource allocation, forecasting, spend analysis
Education OS Core	Entire institution	Identity, RBAC, event bus, integrations, analytics, audit, compliance

Vendor comparison

A useful framing is that EAB, Ellucian, and HEM each consolidate different parts of the stack rather than covering the same center of gravity. EAB is strongest in coordinated care and outreach workflows, Ellucian is strongest in end-to-end SIS plus finance, and HEM emphasizes admissions, student records, communication, and finance in one cloud platform. [2] [1] [3]

Capability	EAB Navigate360	Ellucian	HEM
Central student profile	Complete Student Profile with SIS/LMS/engagement signals [3] [5]	Centralized student information and lifecycle support [1] [6]	Single profile centralizing admissions, academics, and records [2]
Advising and care network	Coordinated Care Network, cases, referrals, appointments, alerts [3]	Advising, degree planning, academic progress, self-service [1] [7]	Supports student dashboards and staff workflows, but advising depth is less explicit in the source [2]
Registration and academic operations	CRM-adjacent support, holds center, journeys, planning signals [3]	Strong SIS core for enrollment, registration, course planning, records [1] [6]	Admissions, schedules, attendance, grades, records in one system [2]
Faculty workflow support	Faculty progress reports and student success reporting [3] [8]	Faculty/student self-service and advising profile views [7]	Staff role permissions and reporting, less detailed faculty workflow description in the source [2]
Communications	Campaigns, templates, two-way SMS, AI assistant, automated outreach [3] [5]	Proactive notifications and student communication support [4]	Multichannel communication, browser calling, digital contracts [2]
Finance and budgeting	Financial planner and aid coordination workflows, but not a full finance suite in the cited material [3] [9]	Budgets, grants, spending, finance self-service, compliance [1] [4] [7]	Finances included in SIS positioning, but budget-planning depth is not detailed in the source [2]
Best-fit model	Student success CRM overlay	Unified SIS + finance backbone	All-in-one enrollment/records cloud platform

Shared identity and RBAC

The shared identity model should treat a person as one canonical institutional subject who may hold multiple roles over time and at the same time. This matches the vendor pattern of connecting prospect, student, faculty, and staff workflows around centralized profiles and cross-system context. [5] [1] [2]

Identity model

```
Person
  ├─ UserAccount
  ├─ RoleAssignments
  ├─ Affiliations
  ├─ LifecycleState
  ├─ Profiles
```

```
  ├─ Credentials
  └─ Consent / policy records
```

RBAC design

Entity	Purpose
`persons`	Canonical human identity
`user_accounts`	Login, MFA, SSO linkage
`roles`	Student, applicant, faculty, advisor, registrar, dean, finance analyst, etc.
`role_assignments`	Scoped role grants by organization, term, and date range
`permissions`	Atomic actions such as `profile.read_private`, `enrollment.approve_override`, `budget.view_department`
`role_permissions`	Default permission bundles
`policy_overrides`	Temporary exceptions and delegated access
`affiliations`	Links to colleges, departments, programs, offices, clubs

Authorization rules

- Use person identity as the root key across all systems.
- Assign roles with scope, such as department, college, program, term, or case.
- Enforce least privilege by default.
- Support delegated and temporary access for coverage, escalation, and substitute advising.
- Separate profile visibility permissions from academic-record permissions.

Complete SQL schema

The SQL below is a practical blueprint for the three layers plus the shared core. Ellucian's positioning around lifecycle, advising, finance, and self-service, together with EAB's emphasis on cases, alerts, and coordinated care, supports a design where student records, workflows, and finance are modeled in one schema family. [7] [1] [3]

```
CREATE TABLE persons (
  id uuid PRIMARY KEY,
  institutional_id text UNIQUE,
  lifecycle_state text NOT NULL CHECK (
    lifecycle_state IN (
      'prospect','applicant','admitted','matriculating','student',
      'faculty','staff','alumni','inactive'
    )
  ),
  legal_first_name text,
  legal_last_name text,
  preferred_name text,
  primary_email text UNIQUE NOT NULL,
```

```
  phone text,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE user_accounts (
  id uuid PRIMARY KEY,
  person_id uuid NOT NULL UNIQUE REFERENCES persons(id),
  username text UNIQUE NOT NULL,
  sso_subject text UNIQUE,
  mfa_enabled boolean NOT NULL DEFAULT false,
  account_status text NOT NULL CHECK (
    account_status IN ('pending','active','locked','disabled')
  ),
  last_login_at timestamptz
);

CREATE TABLE organizations (
  id uuid PRIMARY KEY,
  parent_org_id uuid REFERENCES organizations(id),
  org_type text NOT NULL CHECK (
    org_type IN ('institution','college','school','department','office','program','cl
  ),
  name text NOT NULL,
  code text UNIQUE
);

CREATE TABLE roles (
  id uuid PRIMARY KEY,
  key text UNIQUE NOT NULL,
  name text NOT NULL
);

CREATE TABLE permissions (
  id uuid PRIMARY KEY,
  key text UNIQUE NOT NULL,
  description text NOT NULL
);

CREATE TABLE role_permissions (
  role_id uuid NOT NULL REFERENCES roles(id),
  permission_id uuid NOT NULL REFERENCES permissions(id),
  PRIMARY KEY (role_id, permission_id)
);

CREATE TABLE role_assignments (
  id uuid PRIMARY KEY,
  person_id uuid NOT NULL REFERENCES persons(id),
  role_id uuid NOT NULL REFERENCES roles(id),
  organization_id uuid REFERENCES organizations(id),
  scope_type text NOT NULL CHECK (
    scope_type IN ('global','organization','program','term','case','student')
  ),
  scope_id uuid,
  starts_at timestamptz,
  ends_at timestamptz,
```

```
  granted_by uuid REFERENCES persons(id)
);

CREATE TABLE profiles (
  id uuid PRIMARY KEY,
  person_id uuid NOT NULL UNIQUE REFERENCES persons(id),
  profile_slug text UNIQUE,
  headline text,
  bio text,
  photo_url text,
  major_label text,
  class_year text,
  default_visibility text NOT NULL CHECK (
    default_visibility IN ('public','campus','connections','private')
  ),
  portfolio_enabled boolean NOT NULL DEFAULT true,
  messaging_enabled boolean NOT NULL DEFAULT true,
  updated_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE profile_fields (
  id uuid PRIMARY KEY,
  profile_id uuid NOT NULL REFERENCES profiles(id),
  field_key text NOT NULL,
  field_value jsonb NOT NULL,
  UNIQUE(profile_id, field_key)
);

CREATE TABLE profile_visibility_rules (
  id uuid PRIMARY KEY,
  profile_id uuid NOT NULL REFERENCES profiles(id),
  field_key text NOT NULL,
  audience_type text NOT NULL CHECK (
    audience_type IN ('public','campus','connections','faculty','advisor','org','priv
  ),
  audience_value text,
  can_view boolean NOT NULL DEFAULT true
);

CREATE TABLE portfolio_items (
  id uuid PRIMARY KEY,
  profile_id uuid NOT NULL REFERENCES profiles(id),
  item_type text NOT NULL CHECK (
    item_type IN ('coursework','project','credential','award','publication','experien
  ),
  title text NOT NULL,
  description text,
  evidence_url text,
  media_url text,
  related_course_id uuid,
  visibility text NOT NULL CHECK (
    visibility IN ('public','campus','connections','private')
  ),
  verification_status text NOT NULL CHECK (
    verification_status IN ('self_added','faculty_verified','institution_verified')
  ),
```

```
  sort_order integer NOT NULL DEFAULT 0
);

CREATE TABLE terms (
  id uuid PRIMARY KEY,
  code text UNIQUE NOT NULL,
  name text NOT NULL,
  starts_on date NOT NULL,
  ends_on date NOT NULL
);

CREATE TABLE programs (
  id uuid PRIMARY KEY,
  organization_id uuid REFERENCES organizations(id),
  code text UNIQUE NOT NULL,
  name text NOT NULL,
  credential_level text NOT NULL
);

CREATE TABLE students (
  person_id uuid PRIMARY KEY REFERENCES persons(id),
  current_program_id uuid REFERENCES programs(id),
  catalog_term_id uuid REFERENCES terms(id),
  academic_level text,
  status text NOT NULL CHECK (
    status IN ('active','leave','graduated','withdrawn')
  )
);

CREATE TABLE faculty_assignments (
  id uuid PRIMARY KEY,
  person_id uuid NOT NULL REFERENCES persons(id),
  organization_id uuid REFERENCES organizations(id),
  assignment_type text NOT NULL CHECK (
    assignment_type IN ('faculty','advisor','mentor','chair','instructor')
  )
);

CREATE TABLE course_catalog (
  id uuid PRIMARY KEY,
  organization_id uuid REFERENCES organizations(id),
  course_code text UNIQUE NOT NULL,
  title text NOT NULL,
  credits numeric(4,2) NOT NULL
);

CREATE TABLE course_sections (
  id uuid PRIMARY KEY,
  course_id uuid NOT NULL REFERENCES course_catalog(id),
  term_id uuid NOT NULL REFERENCES terms(id),
  section_code text NOT NULL,
  instructor_person_id uuid REFERENCES persons(id),
  seat_capacity integer,
  modality text
);
```

```
CREATE TABLE enrollments (
  id uuid PRIMARY KEY,
  student_person_id uuid NOT NULL REFERENCES students(person_id),
  section_id uuid NOT NULL REFERENCES course_sections(id),
  enrollment_status text NOT NULL CHECK (
    enrollment_status IN ('planned','registered','waitlisted','dropped','completed')
  ),
  grade text,
  registered_at timestamptz
);

CREATE TABLE applications (
  id uuid PRIMARY KEY,
  person_id uuid NOT NULL REFERENCES persons(id),
  applying_term_id uuid REFERENCES terms(id),
  applying_program_id uuid REFERENCES programs(id),
  application_status text NOT NULL CHECK (
    application_status IN ('draft','submitted','review','admitted','denied','waitlis
  ),
  submitted_at timestamptz
);

CREATE TABLE advisor_assignments (
  id uuid PRIMARY KEY,
  student_person_id uuid NOT NULL REFERENCES students(person_id),
  advisor_person_id uuid NOT NULL REFERENCES persons(id),
  assignment_type text NOT NULL CHECK (
    assignment_type IN ('primary','secondary','faculty','success_coach')
  ),
  starts_at timestamptz NOT NULL,
  ends_at timestamptz
);

CREATE TABLE advising_cases (
  id uuid PRIMARY KEY,
  student_person_id uuid NOT NULL REFERENCES students(person_id),
  opened_by_person_id uuid REFERENCES persons(id),
  assigned_to_person_id uuid REFERENCES persons(id),
  case_type text NOT NULL CHECK (
    case_type IN ('academic','financial','wellbeing','conduct','transition','registra
  ),
  priority text NOT NULL CHECK (priority IN ('low','medium','high','urgent')),
  status text NOT NULL CHECK (
    status IN ('open','in_progress','waiting_student','resolved','closed','escalated
  ),
  summary text NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now(),
  closed_at timestamptz
);

CREATE TABLE advising_alerts (
  id uuid PRIMARY KEY,
  student_person_id uuid NOT NULL REFERENCES students(person_id),
  source_type text NOT NULL CHECK (
    source_type IN ('faculty','lms','attendance','registration','finance','self_repo
  ),
```

```
  severity text NOT NULL CHECK (severity IN ('info','warning','critical')),
  alert_key text NOT NULL,
  payload jsonb NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now(),
  case_id uuid REFERENCES advising_cases(id)
);

CREATE TABLE advising_tasks (
  id uuid PRIMARY KEY,
  case_id uuid REFERENCES advising_cases(id),
  assigned_to_person_id uuid REFERENCES persons(id),
  task_type text NOT NULL,
  status text NOT NULL CHECK (
    status IN ('open','in_progress','completed','cancelled')
  ),
  due_at timestamptz,
  metadata jsonb NOT NULL DEFAULT '{}'::jsonb
);

CREATE TABLE appointments (
  id uuid PRIMARY KEY,
  student_person_id uuid REFERENCES students(person_id),
  staff_person_id uuid REFERENCES persons(id),
  appointment_type text NOT NULL,
  starts_at timestamptz NOT NULL,
  ends_at timestamptz NOT NULL,
  status text NOT NULL CHECK (
    status IN ('scheduled','completed','cancelled','no_show')
  ),
  location text,
  notes jsonb
);

CREATE TABLE holds (
  id uuid PRIMARY KEY,
  student_person_id uuid NOT NULL REFERENCES students(person_id),
  hold_type text NOT NULL CHECK (
    hold_type IN ('registration','financial','advising','health','conduct')
  ),
  status text NOT NULL CHECK (
    status IN ('active','resolved','waived')
  ),
  placed_by_person_id uuid REFERENCES persons(id),
  reason text NOT NULL,
  starts_at timestamptz NOT NULL DEFAULT now(),
  ends_at timestamptz
);

CREATE TABLE charges (
  id uuid PRIMARY KEY,
  student_person_id uuid NOT NULL REFERENCES students(person_id),
  term_id uuid REFERENCES terms(id),
  charge_type text NOT NULL,
  amount numeric(12,2) NOT NULL,
  status text NOT NULL CHECK (
    status IN ('pending','posted','paid','reversed','written_off')
```

```
  ),
  posted_at timestamptz
);

CREATE TABLE aid_awards (
  id uuid PRIMARY KEY,
  student_person_id uuid NOT NULL REFERENCES students(person_id),
  term_id uuid REFERENCES terms(id),
  award_type text NOT NULL,
  amount numeric(12,2) NOT NULL,
  status text NOT NULL CHECK (
    status IN ('offered','accepted','disbursed','cancelled')
  )
);

CREATE TABLE budgets (
  id uuid PRIMARY KEY,
  organization_id uuid NOT NULL REFERENCES organizations(id),
  fiscal_year integer NOT NULL,
  version text NOT NULL,
  status text NOT NULL CHECK (
    status IN ('draft','approved','closed')
  ),
  total_amount numeric(14,2) NOT NULL
);

CREATE TABLE budget_lines (
  id uuid PRIMARY KEY,
  budget_id uuid NOT NULL REFERENCES budgets(id),
  category text NOT NULL,
  subcategory text,
  planned_amount numeric(14,2) NOT NULL,
  actual_amount numeric(14,2) NOT NULL DEFAULT 0
);

CREATE TABLE resource_allocations (
  id uuid PRIMARY KEY,
  organization_id uuid NOT NULL REFERENCES organizations(id),
  budget_line_id uuid REFERENCES budget_lines(id),
  allocation_type text NOT NULL CHECK (
    allocation_type IN ('staffing','technology','facilities','scholarship','programm
  ),
  amount numeric(14,2) NOT NULL,
  justification text
);

CREATE TABLE api_clients (
  id uuid PRIMARY KEY,
  client_key text UNIQUE NOT NULL,
  client_type text NOT NULL CHECK (
    client_type IN ('internal_service','partner_app','integration')
  ),
  active boolean NOT NULL DEFAULT true
);

CREATE TABLE audit_events (
```

```
  id uuid PRIMARY KEY,
  actor_person_id uuid REFERENCES persons(id),
  entity_type text NOT NULL,
  entity_id uuid NOT NULL,
  action text NOT NULL,
  payload jsonb NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now()
);
```

API specifications

The API should be split into shared core services and layer-specific domains. Because the architecture centralizes identity, workflow, and records, APIs should expose role-scoped views over the same entities rather than duplicating data silos. [1] [5]

Core APIs

API	Purpose
`POST /auth/token`	Session and token issuance
`GET /me`	Current person, account, and role context
`GET /me/permissions`	Resolved RBAC permissions
`GET /organizations/:id`	Organization metadata and hierarchy
`GET /audit-events`	Authorized audit search

Student OS APIs

Method	Endpoint	Purpose
`GET`	`/profiles/:slug`	Public/campus profile read with visibility filtering
`PATCH`	`/me/profile`	Update profile basics
`POST`	`/me/portfolio-items`	Add coursework/project/credential showcase item
`GET`	`/me/enrollments`	Current and historical course enrollments
`POST`	`/registration/cart/items`	Add section to registration cart
`POST`	`/registration/submit`	Attempt enroll/swap/drop transaction
`GET`	`/me/holds`	Active student holds
`GET`	`/me/advising`	Advisor assignments, alerts, appointments, tasks

Faculty and Institution OS APIs

Method	Endpoint	Purpose
GET	`/students/:id/360`	Staff student 360 view
POST	`/advising/alerts`	Create early alert
POST	`/advising/cases`	Open advising/support case
PATCH	`/advising/cases/:id`	Update status, priority, assignment
POST	`/appointments`	Create appointment
POST	`/applications/:id/decision`	Record admissions decision
POST	`/holds`	Place institutional hold
GET	`/sections/:id/roster`	Faculty/registrar roster access

Business OS APIs

Method	Endpoint	Purpose
GET	`/finance/budgets`	Budget portfolio list
GET	`/finance/budgets/:id`	Budget detail and line items
POST	`/finance/resource-allocations`	Create allocation record
GET	`/finance/tuition/summary`	Tuition charges, payments, aid, net balance
GET	`/analytics/retention`	Student-success and persistence metrics
GET	`/analytics/resource-utilization`	Department staffing, spend, and program utilization

Example response

```
{
  "studentId": "stu_123",
  "advisorAssignments": [
    {
      "advisorId": "per_19",
      "assignmentType": "primary",
      "name": "Dr. Ana Ruiz"
    }
  ],
  "holds": [
    {
      "holdType": "advising",
      "status": "active",
      "reason": "Pre-registration advising required"
    }
  ],
  "openCases": 1
}
```

Cross-layer workflows

The value of Education OS comes from translating student actions into staff tasks and, when needed, finance or operations actions. EAB's material explicitly describes campaigns, alerts, referrals, and coordinated follow-up triggered from shared information, while Ellucian describes integrated lifecycle and finance processes on one platform.[4] [9]

Workflow examples

Student action	System event	Staff/ops task
Student misses registration step	`registration.window.missed`	Advisor task + reminder campaign
Faculty submits poor progress report	`faculty.progress_flagged`	Advising case created and assigned
Student self-reports financial concern	`support.hand_raise.financial`	Financial aid referral + advisor visibility
Applicant deposits	`admissions.deposit_confirmed`	Orientation, advising assignment, student account provisioning
Student adds course with restriction	`registration.override_requested`	Registrar or advisor approval queue
Department budget variance exceeds threshold	`finance.variance.alert`	Business officer review task

Example event flow

```
Student action
  → Domain event emitted
  → Workflow engine evaluates rules
  → Creates task / case / notification
  → Routes to advisor, faculty, registrar, or finance
  → Staff resolves or escalates
  → Student receives update in Student OS
```

Identity and authorization flow

RBAC should resolve permissions dynamically from role, scope, organization, and lifecycle state. Since vendors increasingly combine SIS, LMS, CRM, and engagement signals into one institutional context, authorization has to support context-aware access rather than one fixed role table.[3] [5]

Permission resolution

```
User authenticates
  → Load person record
  → Load active role assignments
  → Expand role-permission bundles
  → Apply scope filters and temporary overrides
```

```
  → Evaluate resource policy
  → Return allowed actions
```

Sample permission bundles

Role	Example permissions
Student	`profile.update_self`, `registration.submit_self`, `appointment.book_self`
Faculty	`roster.read_section`, `alert.create_student`, `note.add_academic`
Advisor	`student360.read_assigned`, `case.manage_assigned`, `hold.view`, `appointment.manage`
Registrar	`registration.override`, `section.manage`, `hold.place_registration`
Finance analyst	`budget.read_org`, `allocation.create_org`, `tuition.summary.read`
Dean	`analytics.read_college`, `budget.approve_college`, `student-risk.aggregate.read`

Recommended blueprint

The best blueprint is:

- Use **Ellucian-like** lifecycle depth for student records, registration, and finance.[4] [1]
- Use **EAB-like** coordinated care, alerts, referrals, campaigns, and staff workflow automation on top of that core.[9] [5] [3]
- Use **HEM-like** simplicity for admissions, communications, dashboards, and all-in-one staff productivity.[2]

That combination creates a credible Semester architecture: a super-app where Student OS, Faculty/Institution OS, and Business OS share one identity model, one event system, and one institutional graph.[1] [2] [3]

⁂

1. https://www.ellucian.com/products/student/student-information-systems
2. https://www.higher-education-marketing.com/student-portal/student-information-system
3. https://eab.com/solutions/navigate360/
4. https://www.ellucian.com/products/student
5. https://eab.com/navigate360-ai/
6. https://www.ellucian.com/blog/what-student-management-system
7. https://www.ellucian.com/assets/en/brochure/brochure-learn-more-about-ellucian-banner-self-service.pdf
8. https://eab.com/resources/video/navigate360-student-crm-for-community-colleges/
9. https://eab.com/resources/blog/student-success-blog/financial-aid-season-even-more-complicated/

10. https://www.ellucian.com/node/1416?field_topic[129]=129